

TrackBit

Developer Knowledge Base

A Comprehensive Guide Based on
"Every Programmer Should Know"

Project: TrackBit - Offline-First Business Management Application
Version: 1.0.0 | Date: July 26, 2026

Source: github.com/mtdvio/every-programmer-should-know

Document generated on July 26, 2026
Source knowledge from "Every Programmer Should Know" by mtdvio (CC-BY-4.0 License)

Table of Contents

1. Introduction
2. Security
3. Distributed Systems
4. Architecture
5. Memory Management
6. Floating Point Arithmetic
7. Unicode and Strings
8. Latency and Performance
9. Time and Timezones
10. Code Quality Practices
11. Falsehoods Programmers Believe
12. TrackBit Priority Action Items
13. References

1. Introduction

This document compiles essential software engineering knowledge from the "Every Programmer Should Know" repository (100K+ stars on GitHub) and maps each concept directly to the TrackBit codebase. The goal is to provide actionable guidance for improving code quality, security, reliability, and maintainability.

TrackBit is a Flutter-based, offline-first business management application with:

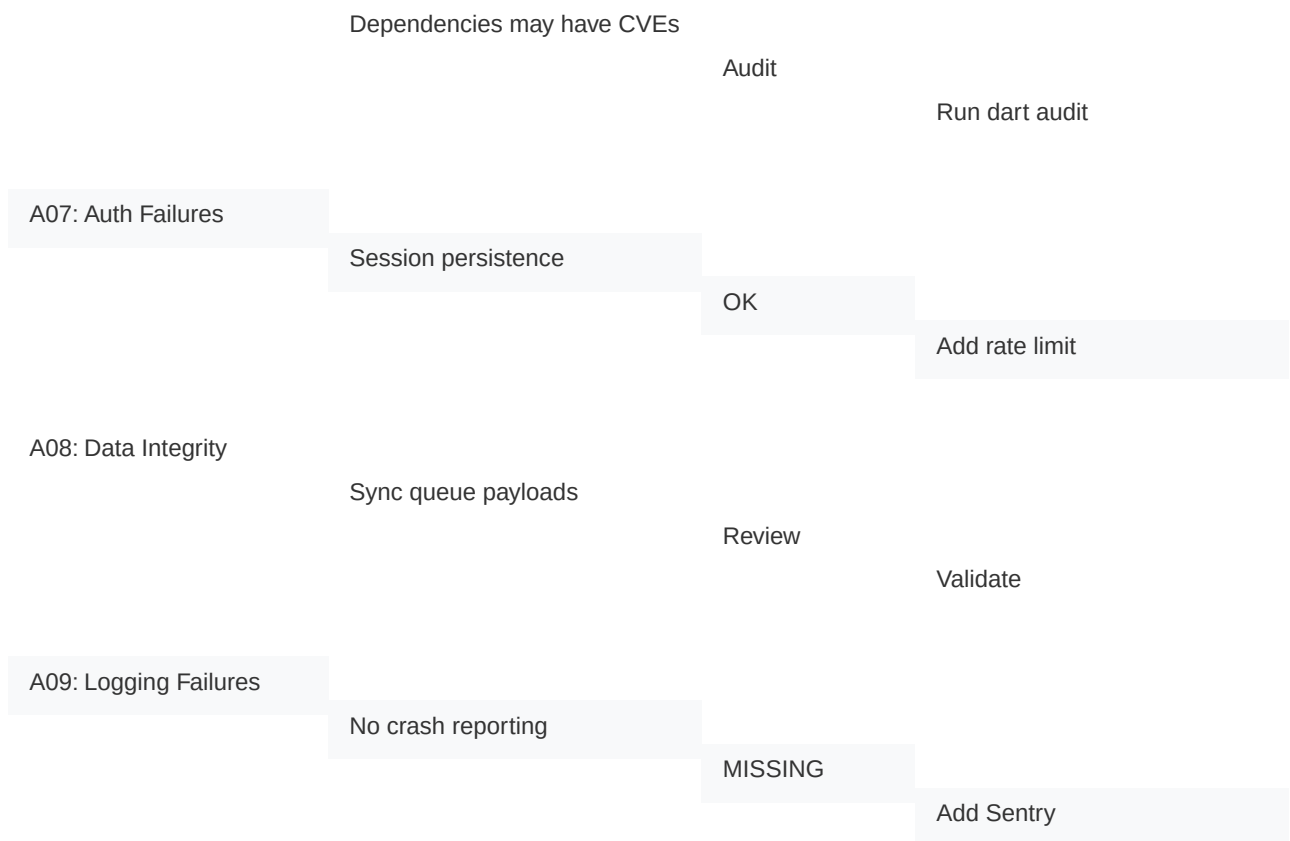
- 21 SQLite (Drift) tables
- 13 feature modules (POS, Inventory, CRM, Sales, Invoicing, Expenses, Employees, Roles, Reports, Sync, Settings, Auth, Dashboard)
- Peer-to-peer data synchronization via WebSocket
- Role-Based Access Control (RBAC) with 43 permissions
- Cross-platform targets: Android, iOS, Web

2. Security

2.1 OWASP Top 10

The OWASP Top 10 is the "gold standard" for web application vulnerabilities. While TrackBit is primarily a mobile/offline app, several OWASP categories are directly relevant:

OWASP Category	Relevance	Status	Action
A01: Broken Access Control	RBAC with 43 permissions	Implemented	Audit checks
A02: Cryptographic Failures	SHA-256 passwords	CRITICAL	Use bcrypt
A03: Injection	Drift ORM queries	Partial	Verify sync SQL
A04: Insecure Design	Peer sync architecture	Review	Add auth
A05: Security Misconfig	Default admin creds	Known	Force change
A06: Vulnerable Components			



2.2 Cryptographic Right Answers

Key principles for
TrackBit:

1. Password Hashing:
SHA-256 is NOT
designed for password
hashing. It is too fast,
making brute-force
attacks feasible. Use
bcrypt, scrypt, or argon2
instead.
2. Encryption at Rest:
SQLite database is
stored unencrypted on
device. Consider
SQLCipher for sensitive
business data.
3. Encryption in Transit:
WebSocket sync uses

wss:// (good), but ensure certificate pinning is implemented.

4. Random Number

Generation: Use

`dart:crypto`

`SecureRandom` for tokens, not `Random()`.

2.3 Rolling Your Own Crypto

TrackBit should NEVER implement custom cryptographic algorithms.

The codebase correctly uses: `crypto` package for SHA-256 (though this should be upgraded), `flutter_secure_storage` for key storage, and `web_socket_channel` for encrypted transport.

Rule: Always use well-tested, established cryptographic libraries.

3. Distributed Systems

3.1 Time, Clocks and Ordering of Events (Lamport)

TrackBit's peer-to-peer sync is a distributed system. When two peers modify the same record independently, conflicts arise. Lamport's paper establishes that happens-before relationship must be maintained and logical clocks can establish ordering without physical time synchronization.

Current TrackBit Sync Behavior:

- Uses sync_queue table with status (PENDING/SYNCED)
- Timestamps are used for ordering
- No vector clocks or conflict resolution strategy

Recommended Improvement: Add vector clock to sync entries for proper conflict resolution.

3.2 There is No Now

Clock skew between peer devices means "now" is different for each device. TrackBit's sync assumes synchronized clocks, which is dangerous. Order timestamps may be inconsistent across peers, and last-write-wins conflict resolution may choose wrong version.

Mitigation:

- Use NTP-synchronized time when available
- Add clock skew tolerance window (e.g., 5 seconds)
- Prefer vector clocks over physical timestamps

3.3 Fallacies of Distributed Computing

Fallacy	TrackBit Status	Risk
The network is reliable	WebSocket can disconnect	HIGH
Latency is zero	Sync may be slow on LAN	MEDIUM
Bandwidth is infinite	Large payloads (images)	MEDIUM

The network is secure

wss:// used

LOW

Topology doesn't change

Peers join/leave

HIGH

There is one administrator

Single-user app

LOW

Transport cost is zero

UDP broadcast every 3s

LOW

The network is homogeneous

Mixed Android/iOS/Web

MEDIUM

4. Architecture

4.1 Out of the Tar Pit

The paper argues that complexity arises from accidental state management. TrackBit's Clean Architecture helps, but entities should be immutable (currently they are not), state should be managed through pure functions where possible, and side effects should be isolated in the data layer (already done).

4.2 CQRS and Event Sourcing

TrackBit's sync queue is already an event-sourcing-like pattern: changes are logged as events in `sync_queue`, events are applied to peers in order, and events can be replayed. Consider formalizing the event schema and adding event versioning for backward compatibility.

4.3 Practical Object Oriented Design

Key principles for TrackBit: Single Responsibility (each repository handles one entity type), Open/Closed (feature modules should be extensible without modification), and Dependency Inversion (abstract repositories in domain layer, concrete in data layer). TrackBit follows these principles well through Clean Architecture.

5. Memory Management

While Dart has garbage collection, memory leaks still occur through unclosed stream subscriptions, retained references in providers, and unclosed database connections.

TrackBit Risk Areas:

- WebSocket connections in `web_socket_sync_service.dart`
- mDNS listeners in `peer_discovery_service.dart`
- Stream controllers in state notifiers
- Database connections (Drift handles this, but verify)

Audit Checklist:

- & All StreamSubscription objects are cancelled in dispose()
- & All Riverpod providers use autoDispose where appropriate
- & WebSocket connections are closed when sync stops
- & mDNS socket is closed when discovery stops
- & No circular references between providers

6. Floating Point Arithmetic

CRITICAL FOR TRACKBIT: The app handles money (prices, totals, tax, discounts) using double type, which has floating-point precision issues.

Example: $0.1 + 0.2 = 0.30000000000000004$

Current TrackBit money fields using double:

- Products.price, Products.cost
- OrderItems.unitPrice, OrderItems.subtotal
- Orders.subtotal, tax, discount, total
- Payments.amount
- Expenses.amount
- Employees.salary

Solution Options:

1. Integer Cents (Recommended for simplicity): Store all money as int (cents). Display: amount ~/ 100 dollars, amount % 100 cents.
2. Decimal Package (Recommended for precision): Use decimal package for exact decimal arithmetic.

Recommended Approach: Integer cents for simplicity, or decimal package for readability.

7. Unicode and Strings

TrackBit stores product names, customer names, addresses, and notes that may contain international characters. Search may fail with Unicode normalization, sorting may be incorrect, and string length calculations may be wrong.

Homoglyphs Risk:

Homoglyphs are characters that look alike but are different (e.g., "a" Cyrillic vs "a" Latin). This affects SKU validation, barcode scanning, and usernames. Validate input characters against a whitelist for SKUs and barcodes.

8. Latency and Performance

TrackBit Performance Targets:

- POS checkout: < 200ms (user expectation)
- Product search: < 100ms
- Database queries: < 50ms
- Sync message round-trip: < 2s on LAN

Optimization Strategies:

- Use database indexes on frequently queried columns
- Implement pagination for large lists (products, orders)
- Cache frequently accessed data (categories, settings)
- Use StreamProvider for real-time UI updates

9. Time and Timezones

TrackBit stores timestamps for order creation, expense recording, sync operations, employee hire dates, and invoice due dates. If peers are in different timezones, timestamps may conflict.

Best Practice:

- Always store UTC: Never store local time
- Convert on display: Convert UTC to local only when showing to user
- Handle DST: Daylight Saving Time causes ambiguity
- Clock skew: Devices may have different times

10. Code Quality Practices

Current TrackBit Test Coverage:

- 39 tests across 5 test files
- 10 of 13 features have NO tests
- No integration tests
- No code coverage tracking

Target: 200+ tests covering all features, with 80%+ coverage.

11. Falsehoods Programmers Believe

Falsehood	TrackBit Impact	Mitigation
Names are always ASCII	Customer/product names	Use Unicode support
Phone numbers are numbers	Customer phone field	Store as string
Email addresses are simple	Customer email	Use proper validation
Addresses have fixed format	Customer/business address	Use flexible fields
Dates are always same format	International usage	Use ISO 8601 (UTC)
Money is always positive	Expenses could be negative	Validate at business level

12. TrackBit Priority Action Items

P0 - Critical (Do Immediately)

#	Action	Source	Files
1	Replace SHA-256 with bcrypt	Crypto Answers	auth_repository_impl.dart
2	Replace double money with cents	Floating Point	All entities
3	Audit SQL injection in sync	OWASP A03	sync_service.dart
4	Force password change for admin	OWASP A05	main.dart

P1 - High Priority (This Week)

#	Action
5	Add vector clocks to sync

6

Audit stream/provider disposal

7

Normalize Unicode for search

8

Add Sentry crash reporting

13. References

Primary Source

- Every Programmer Should Know: github.com/mtdvio/every-programmer-should-know

Security

- OWASP Top 10: owasp.org/www-project-top-ten
- Cryptographic Right Answers: gist.github.com/tqbf/be58d2d39690c3b366ad
- Rolling Your Own Crypto: loup-vaillant.fr/articles/rolling-your-own-crypto

Distributed Systems

- Time, Clocks and Ordering of Events: Lamport, 1978
- There is No Now: queue.acm.org/detail.cfm?id=2745385
- Fallacies of Distributed Computing: pages.cs.wisc.edu/~zuyu/files/fallacies.pdf
- Designing Data-Intensive Applications: Martin Kleppmann

Architecture

- Out of the Tar Pit: github.com/papers-we-love/papers-we-love
- CQRS and Event Sourcing: youtube.com/watch?v=JHGkaShoyNs
- Practical Object Oriented Design: poodr.com

Memory and Performance

- What Every Programmer Should Know About Memory: lwn.net/Articles/250967/
- Latency Numbers Every Programmer Should Know: gist.github.com/jboner/2841832

Numbers and Strings

- The Floating Point Guide: floating-point-gui.de
- What Every Computer Scientist Should Know About Floating-Point: docs.oracle.com
- Unicode and Character Sets: joelonsoftware.com/articles/Unicode.html

Code Quality

- Clean Code: Robert C. Martin
- Test Driven Development: Kent Beck
- Working Effectively with Legacy Code: Michael Feathers
- The Art of Readable Code: Dustin Boswell & Trevor Foucher

Source knowledge from "Every Programmer Should Know" by mtdvio (CC-BY-4.0 License).